AI DevCoach — 개인별 학습진도관리 상세 화면설계서

화면설계서 v1.0 | 기능 ID: FR-04 | 연계: FR-02 수준 진단, FR-03 학습계획, FR-05 GitHub 분석, FR-08 성과 분석

1. 화면 설계 개요

1.1 개발 목적

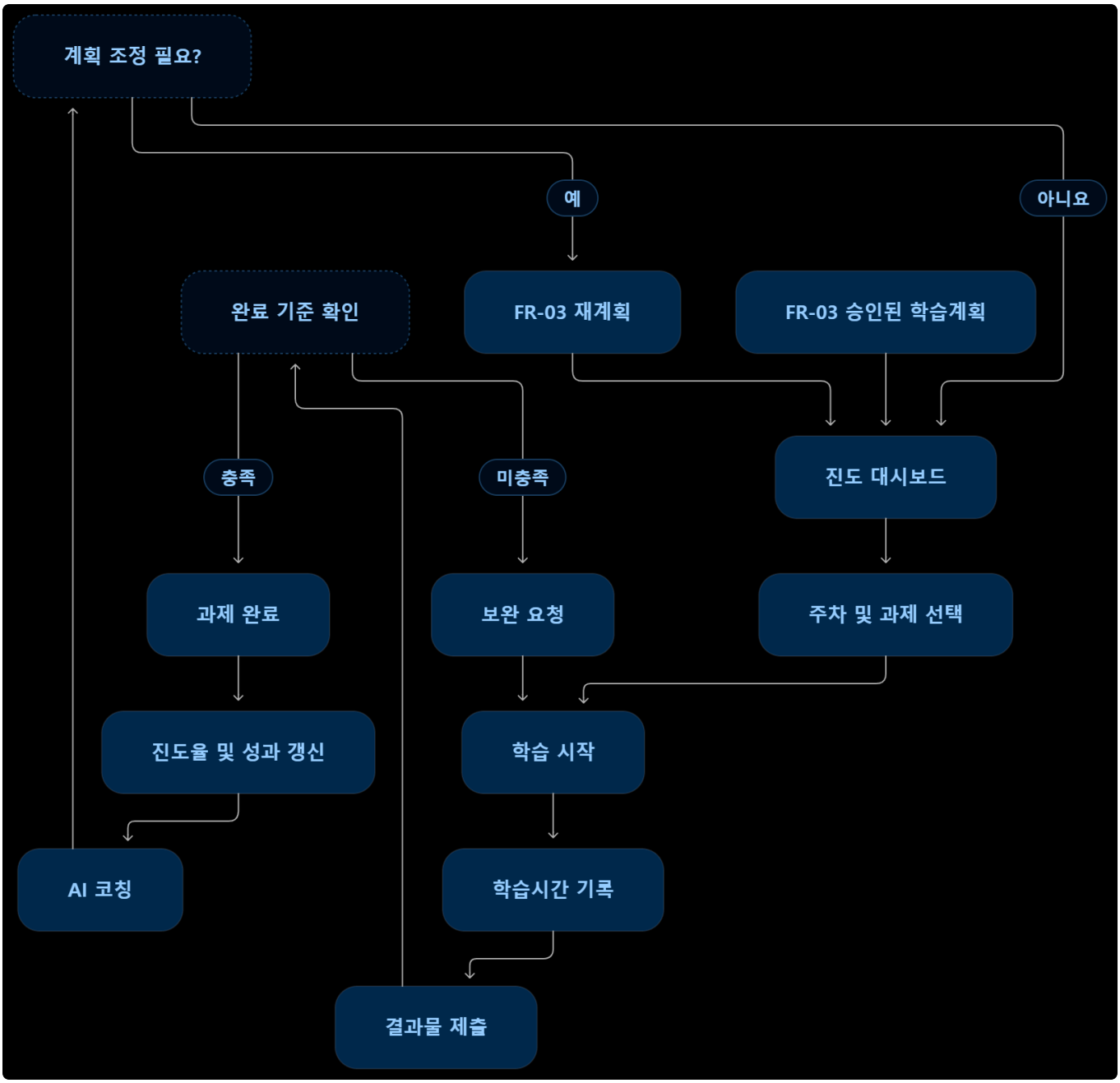
학습자가 AI DevCoach에서 수립한 개인별 학습계획을 기준으로 주차별·과제별 학습 진행 상황을 확인하고, 학습 결과물을 제출하며, 계획 대비 실적을 관리할 수 있는 화면을 구현합니다.

AI Agent는 학습 이력과 미완료 과제를 분석하여 다음 학습 행동을 제안합니다. 단, 과제 완료 여부와 실제 학습시간은 검증 가능한 기록을 기준으로 관리합니다.

1.2 핵심 기능

기능 ID	기능명	주요 내용
PM-01	학습 현황 대시보드	전체 진도율, 완료 과제, 학습시간, 주간 목표
PM-02	주차별 진도 조회	주차별 계획과 실제 수행 현황
PM-03	학습 과제 관리	시작, 진행, 완료, 보류 상태 관리
PM-04	학습시간 기록	과제별 실제 학습시간 입력 및 수정
PM-05	결과물 제출	GitHub URL 또는 학습 결과 설명 등록
PM-06	AI 학습 코칭	지연·취약 영역 분석 및 다음 행동 추천
PM-07	학습 이력 조회	완료 내역, 상태 변경, 피드백 기록
PM-08	학습계획 연계	학습 지연 시 FR-03 재계획 요청

1.3 업무 흐름



2. 화면 목록 및 이동 구조

화면 ID	화면명	주요 기능
UI-15	학습진도 대시보드	전체 KPI, 주차별 진도, AI 코칭
UI-16	주차별 학습현황	주차 목표, 과제 목록, 학습시간
UI-17	과제 상세·실적 등록	상태 변경, 학습시간, 결과물 제출
UI-18	학습 이력 및 통계	주차별 실적, 누적 학습시간, 활동 기록
UI-19	AI 학습 코칭	지연 원인 분석, 우선 과제 추천
UI-20	계획 조정 요청	미완료 과제 기반 재계획 연계

3. UI-15~17 인터랙티브 화면 시안

다음 시안은 4주, 주당 8시간, 총 32시간의 예시 학습계획을 사용합니다. 과제 상태, 실제 학습시간, 결과물 URL을 변경하면 화면의 진도율과 통계가 즉시 갱신됩니다.



4. UI-15 학습진도 대시보드 상세 명세

4.1 KPI 표시 항목

항목 ID	항목명	계산·표시 방식
DB-01	전체 과제 완료율	완료 과제 수 ÷ 전체 과제 수 × 100
DB-02	누적 학습시간	학습시간 기록 합계

항목 ID	항목명	계산·표시 방식
DB-03	계획 학습시간	승인된 과제별 예상시간 합계
DB-04	진행 중 과제	`in_progress` 상태 과제 수
DB-05	보류 과제	`blocked` 상태 과제 수
DB-06	주차별 완료율	해당 주 완료 과제 수 ÷ 해당 주 전체 과제 수 × 100
DB-07	AI 코칭 요약	최근 진도·지연 과제에 근거한 추천

주의: 완료율과 학습시간 달성률은 다른 지표입니다. 학습시간이 많더라도 과제 완료 기준을 충족하지 않았다면 완료율이 올라가지 않습니다.

4.2 상태별 UI 표시 규칙

상태	의미	화면 표시
`todo`	학습 시작 전	시작 전
`in_progress`	학습 수행 중	진행 중
`completed`	완료 기준 충족	완료
`blocked`	학습 진행 장애	보류

학습자가 완료를 요청했더라도 GitHub 결과물 검증이 필요한 과제라면 `completion_requested` 상태를 추가하여 검증 전과 완료 확정을 구분할 수 있습니다.

5. UI-16 주차별 학습현황 화면

5.1 주요 표시 항목

항목	상세 내용
주차 선택	1~N주차 탭
주간 학습 목표	FR-03에서 확정된 목표
주간 계획시간	해당 주 과제의 예상시간 합계
주간 실제시간	실제 학습시간 합계
주간 완료율	완료 과제 수 기준
과제 목록	과제명, 상태, 계획시간, 실제시간
결과물 상태	미제출, 제출, 검증 대기, 검증 완료
주간 코칭	지연·미완료 과제와 다음 행동 제안

5.2 사용자 조작

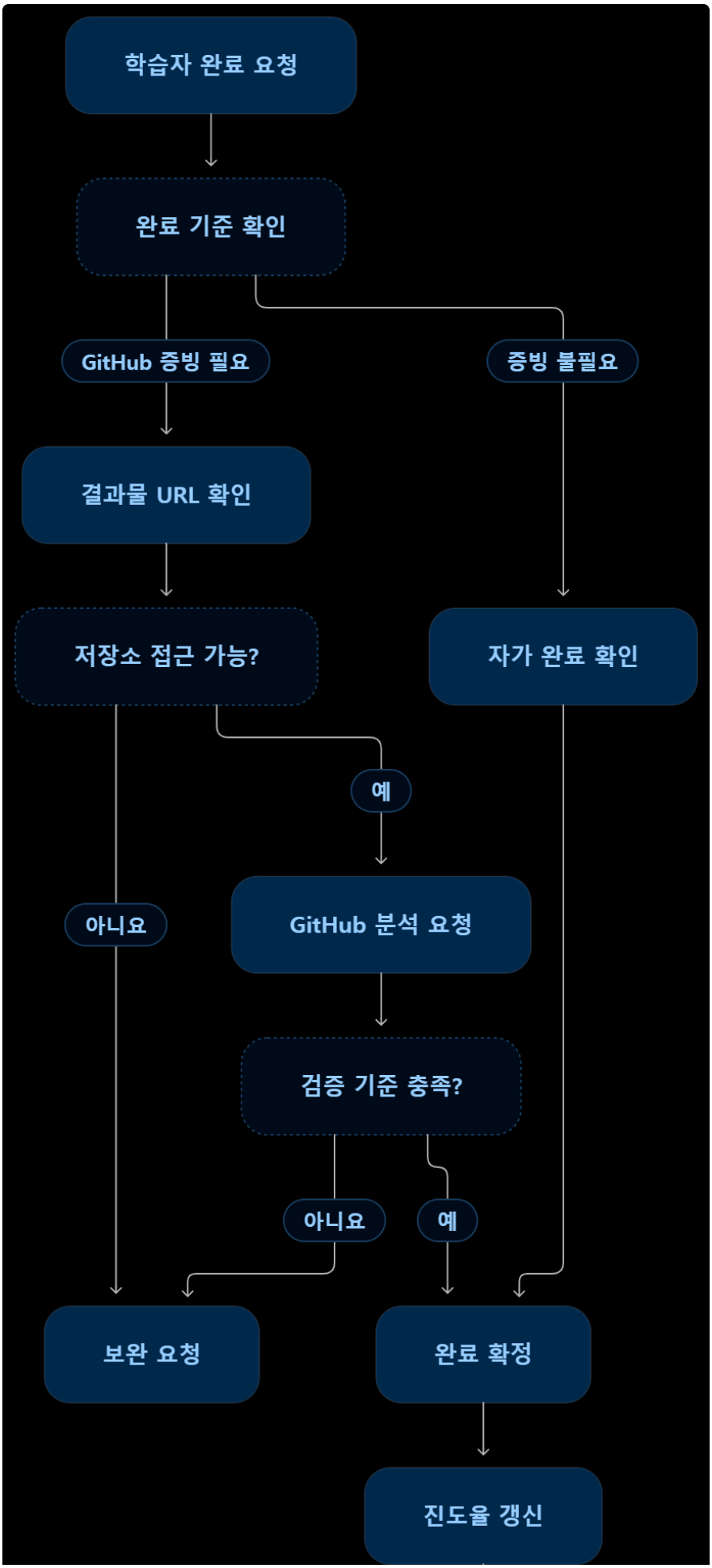
주차를 선택하면 해당 주 과제 목록이 표시되고, 과제를 클릭하면 UI-17 상세 입력 영역이 열립니다. 이전 주차도 조회·수정할 수 있으나, 수정 이력은 별도로 보관합니다.

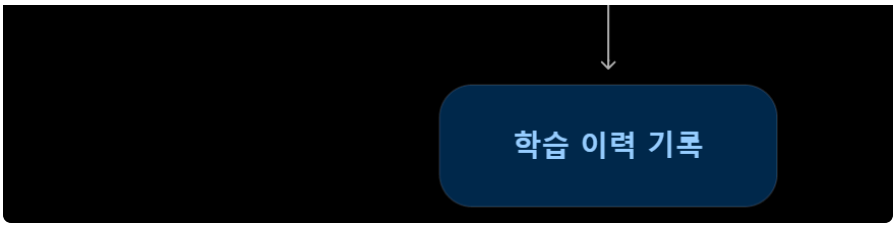
6. UI-17 과제 상세 및 학습실적 등록 화면

6.1 입력 항목

항목 ID	필드명	자료형	필수	검증 규칙
TASK-01	과제 상태	ENUM	Y	허용된 상태값만 입력
TASK-02	학습 시작일	DATE	N	종료일 이후일 수 없음
TASK-03	실제 학습시간	DECIMAL	Y	0 이상, 비정상 값 검증
TASK-04	학습 내용	TEXT	N	최대 2,000자 권장
TASK-05	어려웠던 점	TEXT	N	코칭 분석에 활용
TASK-06	GitHub URL	VARCHAR	조건부	결과물 제출 과제에서 필요
TASK-07	완료 요청	ACTION	조건부	완료 기준 확인 후 처리
TASK-08	학습자 메모	TEXT	N	개인 학습 기록

6.2 과제 완료 처리 로직


학습자 완료 요청
완료 기준 확인
GitHub 증빙 필요
증빙 불필요
결과물 URL 확인
저장소 접근 가능?
자가 완료 확인
예
아니요
GitHub 분석 요청
검증 기준 충족?
아니요
예
보완 요청
완료 확정
진도율 갱신



7. UI-18 학습 이력 및 통계

7.1 통계 지표

지표	계산 방법	활용 목적
과제 완료율	완료 과제 / 전체 과제	학습 진행 수준
학습시간 달성률	실제 학습시간 / 계획 학습시간 × 100	계획 대비 투입시간
주차별 진도율	주차별 완료 과제 / 주차별 전체 과제	주간 진행 현황
과제 지연 수	마감일 경과 및 미완료 과제 수	일정 위험 파악
평균 과제 수행시간	완료 과제의 실제시간 평균	계획시간 적정성 검토
결과물 제출률	결과물 제출 과제 / 제출 대상 과제	실습 수행 여부
학습 연속일	실제 학습 기록이 있는 연속 날짜 수	학습 지속성 확인

시간 달성률이 100%를 초과할 수 있으므로, 대시보드 그래프는 100%까지만 표시하더라도 실제 수치는 별도로 유지합니다.

7.2 활동 이력

활동 이력은 다음 정보를 기록합니다.

- 학습 시작·중단·완료
- 실제 학습시간 등록 및 수정
- GitHub 결과물 제출
- 완료 요청 및 검증 결과
- AI 코칭 생성
- 학습계획 변경 요청

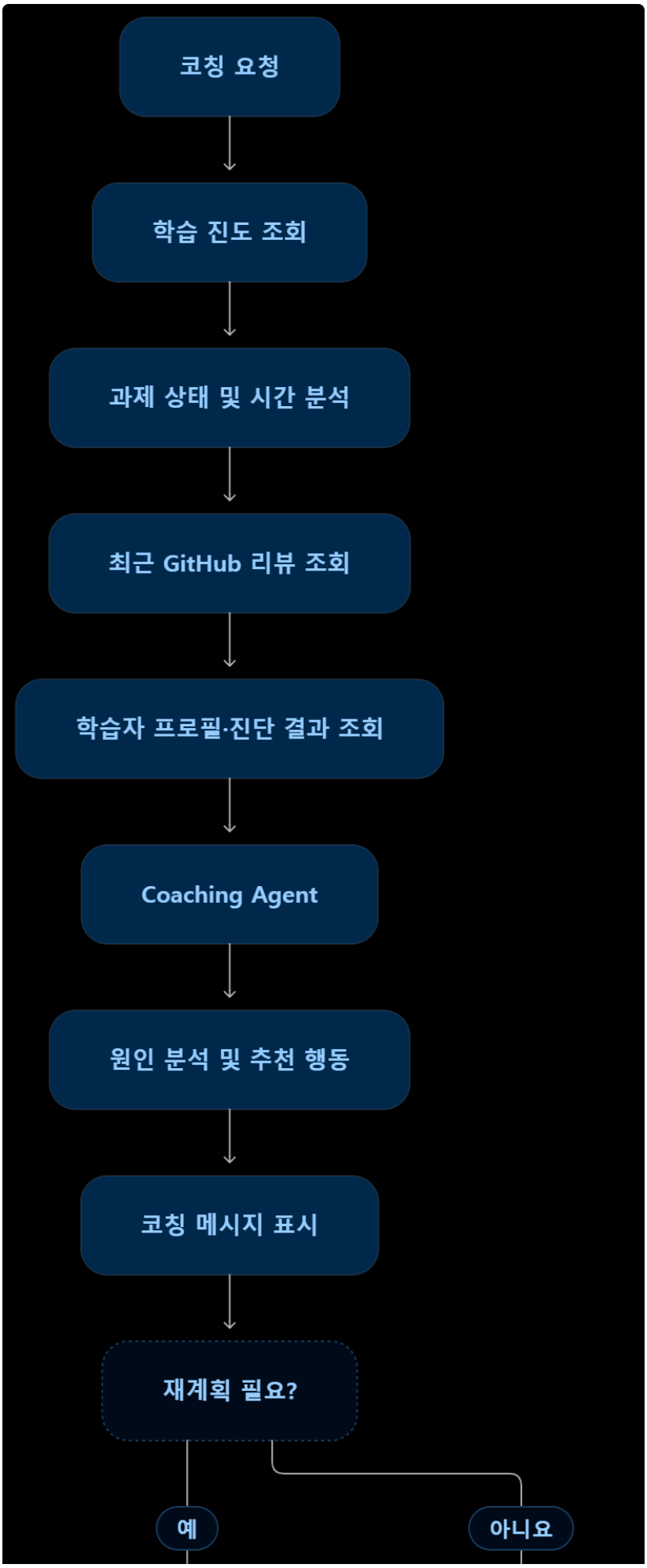
각 이력에는 `user_id`, `task_id`, `event_type`, `event_time`, `old_value`, `new_value`를 저장합니다.

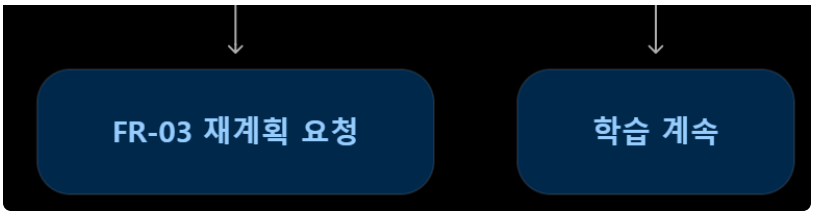
8. UI-19 AI 학습 코칭 설계

8.1 코칭 유형

코칭 유형	발생 조건	AI 제안 내용
학습 지연	마감일 경과 및 미완료	우선 완료할 과제, 일정 조정
학습시간 초과	실제시간이 계획시간 초과	난이도·학습량 재검토
반복 보류	동일 유형 과제 반복 보류	선수 학습·자료 추천
코드 개선 필요	GitHub 리뷰에서 문제 확인	수정 과제와 학습자료
학습 목표 달성	주간 과제 완료	다음 단계 학습 추천
진도 정체	일정 기간 학습 기록 없음	재시작을 위한 작은 과제 제안

8.2 Coaching Agent 처리 구조


코칭 요청
학습 진도 조회
과제 상태 및 시간 분석
최근 GitHub 리뷰 조회
학습자 프로필·진단 결과 조회
Coaching Agent
원인 분석 및 추천 행동
코칭 메시지 표시
재계획 필요?
예
아니요



8.3 Agent Tool 명세

Tool	역할
`get_current_learning_plan`	현재 승인된 학습계획 조회
`get_task_progress`	과제별 상태·실적 조회
`get_learning_time_logs`	학습시간 기록 조회
`get_overdue_tasks`	마감 초과 과제 조회
`get_latest_code_review`	최근 GitHub 코드 리뷰 조회
`generate_coaching_feedback`	AI 코칭 생성
`request_plan_revision`	사용자 승인 기반 재계획 요청

AI 코칭은 조회 데이터에 근거해야 하며, 실제 기록이 없는데 학습을 수행했다고 추정하거나 사용자의 과제 상태를 임의 변경해서는 안 됩니다.

9. 데이터베이스 설계

기존 `learning_plans`, `learning_plan_weeks`, `learning_tasks`를 활용하고, 진도관리용 테이블을 추가합니다.

테이블	주요 필드	용도
`learning_tasks`	task_id, week_id, status, due_date, completed_at	과제 상태
`learning_time_logs`	log_id, task_id, user_id, learning_date, duration_minutes	실제 학습 시간
`task_submissions`	submission_id, task_id, github_url, commit_sha, submitted_at, status	결과물 제출
`task_verifications`	verification_id, submission_id, result, evidence, checked_at	완료 검증
`learning_activity_logs`	activity_id, task_id, event_type, old_value, new_value, created_at	활동 이력
`coaching_feedback`	feedback_id, user_id, plan_id, analysis, recommendation, created_at	AI 코칭 기록

학습시간은 합계 필드 하나만 덮어쓰는 방식보다 날짜별 시간 기록을 누적하는 방식이 적합합니다. 이렇게 하면 주간·월간 통계와 학습 이력의 정확성을 높일 수 있습니다.

10. Backend API 설계

Method	Endpoint	기능
GET	`/api/progress/dashboard`	전체 학습 현황 조회
GET	`/api/progress/weeks/{week_no}`	주차별 진도 조회
GET	`/api/tasks/{task_id}`	과제 상세 조회
PATCH	`/api/tasks/{task_id}`	과제 상태·메모 수정
POST	`/api/tasks/{task_id}/time-logs`	학습시간 등록
GET	`/api/tasks/{task_id}/time-logs`	학습시간 이력
POST	`/api/tasks/{task_id}/submissions`	결과물 제출
POST	`/api/tasks/{task_id}/completion-request`	완료 요청
GET	`/api/progress/statistics`	학습 통계 조회
GET	`/api/progress/activities`	활동 이력 조회
POST	`/api/coach/progress-feedback`	AI 진도 코칭 생성
POST	`/api/plans/{plan_id}/revision-request`	학습계획 재조정 요청

과제 실적 등록 요청 예시

```
{
  "task_id": 3,
  "status": "in_progress",
  "learning_date": "2026-10-24",
  "duration_minutes": 60,
  "learning_notes": "Python requests를 이용해 외부 API를 호출함",
  "difficulty_notes": "HTTP 오류 처리 방법이 어려움"
}
```

진도 대시보드 응답 예시

```
{
  "plan_id": "plan_001",
  "total_tasks": 12,
  "completed_tasks": 2,
  "completion_rate": 16.67,
  "planned_minutes": 1920,
```

```
  "actual_minutes": 360,
  "in_progress_tasks": 1,
  "blocked_tasks": 0,
  "weekly_progress": [
    {
      "week_no": 1,
      "total_tasks": 3,
      "completed_tasks": 2,
      "planned_minutes": 480,
      "actual_minutes": 360
    }
  ]
}
```

11. 테스트 케이스

TC ID	테스트 항목	기대 결과
PM-T01	승인된 학습계획 조회	과제 목록 정상 표시
PM-T02	주차 변경	해당 주차 과제 표시
PM-T03	과제 시작	상태가 `in_progress`로 변경
PM-T04	학습시간 등록	기록 추가 및 누적시간 갱신
PM-T05	과제 완료 요청	완료 기준 검증 절차 실행
PM-T06	GitHub URL 제출	URL 형식 및 접근 가능 여부 확인
PM-T07	완료 기준 미충족	보완 요청 표시
PM-T08	과제 완료 확정	완료율 갱신
PM-T09	학습시간 수정	이력 보존 및 통계 재계산
PM-T10	주간 통계 조회	계획·실적 합계 일치
PM-T11	AI 코칭 요청	실제 진도 데이터 기반 피드백
PM-T12	재계획 요청	FR-03으로 정상 연계
PM-T13	타 사용자 과제 접근	접근 거부
PM-T14	중복 완료 요청	중복 이력·중복 집계 방지

12. 개발 일정 및 역할 분담

개발 작업	담당	목표 공수
대시보드 및 주차별 화면	Frontend	5h

개발 작업	담당	목표 공수
과제 상세·실적 등록 UI	Frontend	4h
학습시간·과제 상태 API	Backend	5h
진도 통계 및 이력 관리	Backend	4h
AI 진도 코칭 연동	AI Agent	4h
통합 테스트 및 오류 수정	공동	4h
합계		26h

26시간은 프로젝트 전체 240시간 안에서 배정하는 세부 목표 공수입니다. 기존 주차별 개발계획의 진도 관리·통합·테스트 작업에 배분하여 관리합니다.

13. 최종 완료 기준

FR-04 개인별 학습진도관리 기능은 다음 사항을 모두 충족해야 합니다.

- 승인된 개인별 학습계획과 과제를 조회할 수 있다.
- 전체·주차별 과제 완료율을 정확하게 계산한다.
- 학습자가 과제 상태와 실제 학습시간을 기록할 수 있다.
- GitHub 결과물 또는 학습 증빙을 제출할 수 있다.
- 완료 기준에 따라 과제 완료를 확인한다.
- 학습시간·과제 변경 이력을 보관한다.
- AI가 학습 진행 상황을 분석하고 다음 행동을 추천한다.
- 학습계획 조정이 필요할 경우 FR-03 재계획 기능과 연결된다.

구현 핵심은 `계획(Plan) → 수행(Task) → 실적(Log) → 검증(Verification) → 코칭(Feedback)` 의 데이터 연결입니다. 이 구조를 확립하면 이후 FR-05 GitHub 분석, FR-06 AI 코드 리뷰, FR-08 학습 성과 분석까지 자연스럽게 확장할 수 있습니다.